

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería en Software

LOGOTIPO
UTEQ

GUÍA Y EVIDENCIA DE EXPOSICIÓN–DEMOSTRACIÓN

Generación de código fuente a partir de modelos UML mediante StarUML

Proyecto Fin de Curso

Sistema de Gestión para un Gimnasio

Subsistema: Gestión de clientes, membresías y pagos

Asignatura: Ingeniería de Requisitos (ISR-401)
Docente: Ing. Gleiston Guerrero Ulloa, PhD.
Periodo académico: 2026–2027 PPA
Paralelo: 4.º Software “A”

Integrantes y roles

Integrante	Rol
Díaz Pontón Steven Santiago	[Rol por definir]
Escudero Plaza María del Rosario	[Rol por definir]
Mera Arias Erick Jhair	[Rol por definir]
Mesías Quijije Jhon Alexander	[Rol por definir]

Repositorio público de GitHub

https://github.com/Erick-Mera/GA_Exposicion_Segundo_Corte

Quevedo – Ecuador
2026

Índice

Resumen	1
1 Marco teórico	1
1.1 Model-Driven Engineering (MDE)	1
1.2 Model-Driven Architecture (MDA)	1
1.3 Model-Driven Development (MDD)	1
1.4 Niveles de abstracción	1
1.4.1 Computation Independent Model (CIM)	1
1.4.2 Platform Independent Model (PIM)	1
1.4.3 Platform Specific Model (PSM)	1
1.5 Transformaciones de modelos	1
1.5.1 Transformaciones modelo a modelo (M2M)	1
1.5.2 Transformaciones modelo a texto (M2T)	1
1.6 Metamodelos	1
1.7 Perfiles UML, estereotipos y valores etiquetados	1
1.8 Plantillas de generación	1
1.9 Forward, reverse y round-trip engineering	1
1.10 Diagramas UML como origen de generación	1
1.11 Estado del arte de las herramientas de transformación	1
2 Justificación de la selección de StarUML	1
2.1 Requisitos y restricciones tecnológicas del PFC	1
2.2 Criterios de selección de la herramienta	1
2.3 Compatibilidad de StarUML con el lenguaje y la plataforma objetivo	1
2.4 Licencia y disponibilidad	2
2.5 Curva de aprendizaje	2
2.6 Comparación con una herramienta alternativa	2
2.7 Decisión de selección	2
3 Descripción del subsistema del PFC	2
3.1 Contexto del Sistema de Gestión para un Gimnasio	2
3.2 Descripción del subsistema de gestión de clientes, membresías y pagos	2
3.3 Actores relacionados	2
3.4 Requisitos funcionales relevantes	2
3.5 Procesos principales	2

3.6	Alcance del subsistema	2
3.7	Exclusiones del subsistema	2
4	Modelo UML de origen	2
4.1	Descripción general del modelo	2
4.2	Diagrama de clases	2
4.3	Clases del dominio	2
4.4	Atributos y operaciones	2
4.5	Relaciones y cardinalidades	2
4.6	Esteretipos y perfiles utilizados	2
4.7	Validación sintáctica y semántica del modelo	2
4.8	Archivo nativo del modelo en el repositorio	2
5	Configuración del generador	2
5.1	Generador o extensión utilizada	3
5.2	Origen y versión del generador	3
5.3	Lenguaje y plataforma objetivo	3
5.4	Parámetros de generación	3
5.5	Directorio de salida y convenciones de nombres	3
5.6	Política ante la regeneración	3
5.7	Decisiones de mapeo UML a código	3
5.8	Archivos de configuración en el repositorio	3
6	Proceso de generación	3
6.1	Preparación del entorno	3
6.2	Apertura y selección del modelo de origen	3
6.3	Ejecución del generador	3
6.4	Registro o log de salida	3
6.5	Árbol del proyecto generado	3
6.6	Artefactos producidos	3
6.7	Tiempo de generación	3
7	Verificación del código generado	3
7.1	Estructura del proyecto generado	3
7.2	Verificación sintáctica	3
7.3	Compilación del código	3
7.4	Ejecución de una unidad mínima demostrable	3

7.5	Correspondencia entre modelo y código	3
7.6	Diferenciación entre código generado y código manual	3
7.7	Limitaciones del generador	4
8	Cierre del ciclo y roundtrip controlado	4
8.1	Estado inicial del modelo y del código	4
8.2	Cambio realizado en el modelo UML	4
8.3	Regeneración del código	4
8.4	Evidencia antes y después	4
8.5	Trazabilidad modelo a código	4
8.6	Manejo del código manual ante la regeneración	4
9	Reparto del trabajo	4
9.1	Roles y responsabilidades	4
9.2	Tareas realizadas por integrante	4
9.3	Commits atribuibles a cada integrante	4
9.4	Log de aportaciones de GitHub	4
9.5	Participación en la exposición y demostración	4
10	Conclusiones	4
10.1	Valor de MDD para el Proyecto Fin de Curso	4
10.2	Aprendizajes del equipo	4
10.3	Limitaciones identificadas	4
	Referencias	4
A	Tabla ampliada de correspondencia modelo–código	5
B	Fragmentos representativos del código generado	5
C	Evidencias complementarias	5
D	Instrucciones detalladas de reproducción	5
E	Evidencia de contribuciones en GitHub	5

Resumen

1. Marco teórico

1.1. Model-Driven Engineering (MDE)

1.2. Model-Driven Architecture (MDA)

1.3. Model-Driven Development (MDD)

1.4. Niveles de abstracción

1.4.1. Computation Independent Model (CIM)

1.4.2. Platform Independent Model (PIM)

1.4.3. Platform Specific Model (PSM)

1.5. Transformaciones de modelos

1.5.1. Transformaciones modelo a modelo (M2M)

1.5.2. Transformaciones modelo a texto (M2T)

1.6. Metamodelos

1.7. Perfiles UML, estereotipos y valores etiquetados

1.8. Plantillas de generación

1.9. Forward, reverse y round-trip engineering

1.10. Diagramas UML como origen de generación

1.11. Estado del arte de las herramientas de transformación

2. Justificación de la selección de StarUML

2.1. Requisitos y restricciones tecnológicas del PFC

2.2. Criterios de selección de la herramienta

2.3. Compatibilidad de StarUML con el lenguaje y la plataforma objetivo

2.4. Licencia y disponibilidad

2.5. Curva de aprendizaje

2.6. Comparación con una herramienta alternativa

2.7. Decisión de selección

3. Descripción del subsistema del PFC

3.1. Contexto del Sistema de Gestión para un Gimnasio

3.2. Descripción del subsistema de gestión de clientes, membresías y pagos

3.3. Actores relacionados

3.4. Requisitos funcionales relevantes

3.5. Procesos principales

3.6. Alcance del subsistema

3.7. Exclusiones del subsistema

4. Modelo UML de origen

4.1. Descripción general del modelo

4.2. Diagrama de clases

4.3. Clases del dominio

4.4. Atributos y operaciones

4.5. Relaciones y cardinalidades

4.6. Estereotipos y perfiles utilizados

4.7. Validación sintáctica y semántica del modelo

4.8. Archivo nativo del modelo en el repositorio

5. Configuración del generador

5.1. Generador o extensión utilizada

5.2. Origen y versión del generador

5.3. Lenguaje y plataforma objetivo

5.4. Parámetros de generación

5.5. Directorio de salida y convenciones de nombres

5.6. Política ante la regeneración

5.7. Decisiones de mapeo UML a código

5.8. Archivos de configuración en el repositorio

6. Proceso de generación

6.1. Preparación del entorno

6.2. Apertura y selección del modelo de origen

6.3. Ejecución del generador

6.4. Registro o log de salida

6.5. Árbol del proyecto generado

6.6. Artefactos producidos

6.7. Tiempo de generación

7. Verificación del código generado

7.1. Estructura del proyecto generado

7.2. Verificación sintáctica

7.3. Compilación del código

7.4. Ejecución de una unidad mínima demostrable

7.5. Correspondencia entre modelo y código

7.6. Diferenciación entre código generado y código manual

7.7. Limitaciones del generador

8. Cierre del ciclo y roundtrip controlado

8.1. Estado inicial del modelo y del código

8.2. Cambio realizado en el modelo UML

8.3. Regeneración del código

8.4. Evidencia antes y después

8.5. Trazabilidad modelo a código

8.6. Manejo del código manual ante la regeneración

9. Reparto del trabajo

9.1. Roles y responsabilidades

9.2. Tareas realizadas por integrante

9.3. Commits atribuibles a cada integrante

9.4. Log de aportaciones de GitHub

9.5. Participación en la exposición y demostración

10. Conclusiones

10.1. Valor de MDD para el Proyecto Fin de Curso

10.2. Aprendizajes del equipo

10.3. Limitaciones identificadas

Referencias

- A. Tabla ampliada de correspondencia modelo–código**
- B. Fragmentos representativos del código generado**
- C. Evidencias complementarias**
- D. Instrucciones detalladas de reproducción**
- E. Evidencia de contribuciones en GitHub**